Menu Scan — Operator Guide

Part 1 — What changed (Phase 4 dish-model rewrite, May 2026)

The menu-scan flow you've used before still works, but the **review UI was rewritten** in May 2026 as part of the dish-model rewrite. The most important changes are below.

Removed

Was	Why it's gone
"Kind" dropdown on each dish (standard / bundle / configurable / course_menu / buffet)	The flat dish model collapses kinds into modifier groups + an optional dining-format hint. Every dish is now a single row.
VariantEditor (the blue "Variants" panel under parent dishes)	Replaced by modifier groups. A "Bundle" with 3 variants becomes a single dish with an "Options" modifier group.
CourseEditor (the purple "Courses" panel for tasting menus)	Replaced by `dining_format = course_menu` + sequential single-choice modifier groups.
Parent/child dish split	All dishes are top-level rows now. Existing parent/variant data still displays on the legacy mobile path until Phase 7.

Added

New	What it does
Modifier Groups Editor (amber panel at the bottom of each dish card)	Replaces variants + courses. One or more groups per dish (Protein, Size, Toppings, etc.). Each group has its own selection rules and a list of options.
Dining Format dropdown (in the field grid where Kind used to be)	Rarely set. Picks a UX hint for special meal styles: buffet, course menu, interactive table, shared plates, sampler. Defaults to "Standard".
Bundled Items editor (emerald panel above modifier groups)	For set menus: lists what comes in the package ("Soup of the day", "Side salad", "Drink").
Transactional confirms	The whole save operation now runs in a single Postgres transaction. If any dish fails to insert, the entire confirm rolls back — no orphan rows.
Restaurant detail view shows modifier groups	Open any restaurant → click a dish row → expand the amber "Modifier groups" disclosure to see the saved structure. Read-only here; edits go through menu-scan review.

What stayed the same

- Upload flow (batch upload images, one job per restaurant)
- Status lifecycle: `pending` → `processing` → `needs_review` → `completed` / `failed`
- Menu-category resolution (existing / canonical / custom)
- Dish-category combobox (filter taxonomy)
- Source-language selector + AI-detection mismatch banner

- Replay (re-run a scan with a different model)
- Skip-restaurant (mark as not needing a menu)
- Admin-bypass create-job action

Part 2 — Full operator guide

Overview

Menu Scan turns photos of a restaurant's menu into structured dish data that consumers can browse in the mobile app. The pipeline is:

1. **You** upload menu photos for a restaurant.
2. **A worker** sends them to OpenAI's vision model (GPT-4o), which returns structured JSON with dishes, categories, modifier options, etc.
3. **The job goes to** `needs_review` with an orange badge — that's your cue.
4. **You** open the job, fix anything the AI got wrong, then click **Save**.
5. **The dishes land in the consumer-facing database**. Mobile users start seeing them after the next CDN cache cycle (~minutes).

When to use it

- A new restaurant has just been onboarded and has no menu yet.
- An existing restaurant has updated their menu and you have fresh photos.
- A previous scan failed or produced low-quality output (use Replay).

When not to use it

- The restaurant has only a handful of dishes — adding them by hand on the restaurant detail page is faster.
- You only want to update one or two dishes — edit them directly on the restaurant detail page (`/restaurants/<id>`) instead of re-scanning the whole menu.

The lifecycle

Status	What it means	Operator action
`pending`	Job created, worker hasn't picked it up yet.	Wait. Usually <60s.
`processing`	Worker is actively calling OpenAI.	Wait. Usually 30–90s.
`needs_review`	AI extraction succeeded. **Your turn to review.**	Open the job, review dishes, click Save.
`completed`	Dishes are in the database.	Nothing more to do. The mobile app will pick them up.
`failed`	Worker errored out. The `last_error` field has the details.	Decide: Replay (re-run), edit input, or flip back to `needs_review` for manual recovery.

Step 1 — Uploading menu images

From the Menu Scan page

1. Go to **Menu Scan** in the admin nav.
2. Click **Upload menu scan** (or use the batch upload form at the top).
3. Pick the restaurant from the dropdown.
4. Add menu page images (PNG/JPG). One image per menu page works best.
5. Click **Submit**.

Image quality tips

- **Good light, flat surface, no glare** — readability matters more than resolution.
- **One page per image** — the AI handles 1–5 pages cleanly. Beyond 5, accuracy drops.
- **High-contrast text** — printed menus work better than handwritten chalkboards.
- **Skip blurry pages** — the AI will hallucinate dishes from unreadable text. Better to scan again with a sharper photo.

Step 2 — Waiting for AI extraction

Jobs are processed by the `menu-scan-worker` Edge Function. You don't need to do anything during this stage — the job list will refresh automatically (or you can hit refresh).

If a job stays in `pending` or `processing` for more than **5 minutes**:

- Click into the job.
- Check the `last_error` field. Common causes: OpenAI rate limit, malformed image, schema mismatch.
- Try **Replay** with `gpt-4o-mini` (cheaper, sometimes more reliable for simple menus).
- If it persists, ping the platform team.

Step 3 — Reviewing dishes (the bulk of your job)

When a job hits `needs_review`, click in. You'll see the **Review Dish Editor** screen.

Top header

- `N to import · M categories` — the count of active (not removed) dishes and the number of menu categories they fall under.
- **Save N dishes** — the big primary button. Disabled while saving.
- **Menu source language** dropdown — used for any custom categories you create. Defaults to the language inferred from the restaurant's country code.
- **AI-detected language banner** — if the AI thinks the menu is in a different language than the country default, a yellow banner appears. Click **Switch to** to override.

Per-dish card

Each dish has its own card. Reading top to bottom:

1. Dish row (top of card)

- **Dish name** input (left)
- **Dining format pill** (violet, only shown if dining_format is set)
- **AI confidence pill** (green ≥85%, yellow ≥60%, red below) — the AI's own estimate of how sure it is about this extraction
- **Page pill** (`pg 1`, `pg 2`) — which source image this dish came from
- **Remove** / **Restore** button — soft-delete a dish from the import

If the AI extracted a dish you don't want, click **Remove** instead of editing it to nothing. Removed dishes stay visible (greyed out) so you can restore them; they're skipped on save.

2. Description textarea

The AI's extracted description. Edit freely. Empty is fine.

3. Field grid (4 columns)

Field	Purpose	When to change
Price	Base dish price (numeric). Empty = price unknown.	Always — AI sometimes flips decimal/thousands separators.
Price label	How the price is presented: `Exact`, `From`, `Per person`, `Market price`, `Ask server`	Set to **From** for dishes where size/options drive the price. Set to **Per person** for buffets and tasting menus.
Primary protein	One of: chicken, beef, pork, lamb, goat, other_meat, fish, shellfish, eggs, vegetarian, vegan	Always required (NOT NULL). The AI usually gets this right; double-check vegan/vegetarian dishes since misclassifying excludes vegan users in the mobile app.
Dining format	`— Standard —` (default), buffet, course_menu, interactive_table, shared_plates, sampler	Set when the meal isn't a regular plated dish. See the dining-format guide below.

4. Menu category dropdown

Three modes:

Mode	What it does	When to use
No category	Dish becomes orphaned — visible on restaurant detail but not under any menu section.	Almost never. Use a custom category instead.
Existing for this restaurant	Attaches the dish to a menu_categories row this restaurant already has.	When re-scanning a menu that's already partially populated.
Canonical taxonomy	Maps to the platform's canonical category list (Mains, Starters, Desserts, etc.). Carries cross-language metadata.	Most of the time — pick from the canonical list when the AI's suggestion matches.
+ Custom name…	You type a category name. Stored as a per-restaurant menu_categories row in the source language.	When the restaurant has a category that doesn't fit the canonical taxonomy (e.g. "Chef's Specials", "Lunch combos").

The AI's verbatim section heading is shown in italics next to the dropdown ("AI saw: 'Starters'") — useful when matching against canonical names.

5. Dish category (filter taxonomy)

The **dish category** is invisible to consumers but powers mobile filtering and recommendations. It's a global taxonomy (e.g. "Pizza", "Burger", "Salad", "Cocktail") — NOT the same as the menu category above.

- **Combobox** — pick from the list.
- **Create inline** button — if the AI suggested something that doesn't exist (e.g. "Birria taco"), use the inline create button to add it on the spot.
- If the AI's suggestion didn't fuzzy-match anything, a yellow warning shows: **"AI suggested 'X' but no close match was found."** Pick or create.

6. Bundled items (emerald panel)

For **set menus**, **combos**, **lunch specials**. Lists what comes in the bundle.

- Click **+ Add item** to add a row.
- Each row: name + optional note (e.g. "tossed", "iced", "house-made").
- These render on the mobile dish card as "comes with: Soup of the day, Side salad (tossed), Drink".

Use for: Set menus, lunch combos, omakase fixed-courses where the customer doesn't pick from options.

Don't use for: Anything where the customer has a choice — that's a modifier group, not a bundled item.

7. Modifier groups (amber panel)

The biggest new piece. Replaces the old variants + courses model entirely. **Use this for any dish where the customer makes a choice.**

Group fields

Field	Purpose
Group name	"Protein", "Size", "Toppings", "Sauce", "Course 1 — Starter"
Selection	`Single` (radio button — one choice) or `Multiple` (checkboxes — many choices). Single locks max to 1.
Min	Minimum selections required to order. `0` = optional, `1` = required, etc.
Max	Maximum allowed. Greyed out when Single (always 1).
Card checkbox	When checked, the option a customer picks for this group surfaces in the dish card name in the mobile feed (e.g. "Pad Thai with Chicken"). Use for **defining** choices (protein, base), not **add-on** choices (sauce, toppings).

Option fields (basic)

Field	Purpose
Option name	"Chicken", "Tofu", "Large", "Extra cheese"
+ delta	Price added when this option is picked. `0` for default-priced, `+3` for an upcharge.
= override	Absolute price replacing the dish base price. Use for **tiered pricing** (e.g. wings 6/12/18). Either delta OR override, not both.
Default checkbox	Pre-selects this option. Affects the feed card's composed name (the default option's name is what shows for new customers).

Option fields (advanced — click ▸ to expand)

Field	When to use
Primary protein override	Set when this specific option introduces a protein different from the dish's base. E.g. Pad Thai base = "chicken" (the default), but the Tofu option overrides to "vegan".
Serves Δ	When this option changes the serving size (e.g. "Large" adds +1 serve). Rarely used.
Removes dietary tags	Toggle the chips for each tag this option **strips**. E.g. "Add cheese" removes vegan; "Add anchovies" removes vegetarian.
Adds allergens	Toggle the chips for each allergen this option **introduces**. E.g. "Add shrimp" adds shellfish; "Add peanuts" adds peanuts.

The advanced panel auto-expands when any of those fields is non-default.

Reordering and removing

- ↑ **/** ↓ buttons reorder groups (and options within a group).
- **×** removes a group or option.

Common patterns

Pattern	How to build it
Pad Thai (pick your protein)	One group named "Protein", Single, min 1, max 1, **Card** checked. Options: Chicken (default, +$0), Tofu (+$0), Shrimp (+$3), Beef (+$2).
Caesar salad (optional add-ons)	One group "Extras", Multiple, min 0, max 4. Options: Anchovies (+$1), Extra parmesan (+$1), Croutons (+$0).
Build your own bowl	Three groups in order: Base (Single, 1/1) → Proteins (Multiple, 1/3) → Toppings (Multiple, 0/4).
Tiered wings (6/12/18)	One group "Quantity", Single, **Card** checked. Options: 6 wings (= $8, default), 12 wings (= $15), 18 wings (= $20). Note: use **override**, not delta.
Tasting menu (3 courses, choose one per)	dining_format = course_menu. Three groups named "Course 1 — Starter", "Course 2 — Main", "Course 3 — Dessert", each Single, min 1, max 1.

Source-language banner

Above the dish list. If the AI detected a language that differs from the restaurant's country default, you get a yellow banner with a quick-switch link. The selected language is used to:

- Set `source_language_code` on any custom categories you create
- Drive the rendering of dropdown labels (existing/canonical category names appear in the picked language when available)

If you confirm without changing the language, custom categories will be stored under the country-default language and won't translate cleanly.

Step 4 — Confirming

Click **Save N dishes**.

- The action runs a Postgres transaction inside the `admin_confirm_menu_scan` RPC.
- **Either all dishes save or none do.** If one dish has invalid data, the whole batch rolls back.
- On success, the job flips to `completed` and you're redirected to the restaurant detail page.

Validation errors

Before sending, the UI checks:

- **Every dish needs a name.** Empty names are rejected. Either fill or remove the dish.
- **Custom category names cannot be empty.** Pick a category or type one in.
- **Every modifier group needs a name, and every option needs a name.** Empty group/option names are rejected.

If the save itself fails (server-side error), a red alert at the top of the page describes what happened. Common server errors:

Message	Meaning	Fix
`NOT_FOUND`	The job no longer exists (deleted by another admin).	Refresh the job list.
`ALREADY_COMPLETED`	Someone else confirmed this job already.	Refresh — the dishes are already in.
`RESTAURANT_NOT_FOUND`	The restaurant was deleted between scan and confirm.	Contact the platform team.
`INVALID_CATEGORY_ID`	A menu_category referenced by a dish doesn't belong to this restaurant.	Refresh — likely a stale category ID.
`INVALID_DISH_CATEGORY_ID`	A dish_category_id no longer exists.	Refresh and reselect.

Edge cases & advanced workflows

A job is stuck in `processing`

Worker probably crashed or the OpenAI call timed out. From the job detail page:

1. Click **Flip to failed** in the debug controls.
2. Click **Replay** with `gpt-4o-mini` (faster, cheaper).
3. If that also fails, check `last_error` and ping platform.

A job came back wrong — Replay

The **Replay** button on the job detail page creates a brand-new pending job with the same input but optionally a different model:

Model	When to pick
`gpt-4o-2024-11-20` (default)	Best accuracy. Use for complex menus, tasting menus, anything with modifiers.
`gpt-4o-mini`	Faster + cheaper. Use for simple menus (≤20 dishes, no modifiers) or when you've burned through the daily 4o budget.

The original (failed) job stays in the list. Both new + old refer to the same uploaded images, so you don't re-upload anything.

Skip a restaurant entirely

Some restaurants don't have a menu to scan (e.g. omakase-only places, food trucks with rotating daily specials). To stop them showing up in your scan queue:

- On the restaurant picker, click **Skip menu scan** next to the row.
- The restaurant gets `skip_menu_scan = true` and disappears from the queue.
- Reversible: go to the restaurant detail page and un-tick the flag.

Admin-bypass create-job (rare)

If you need to manually trigger a scan for a restaurant outside the normal flow:

- Use **adminCreateMenuScanJob** via the menu-scan page's manual upload.
- Provide an array of `menu-scan-uploads` bucket paths and page numbers.
- This is the same path as a normal upload but lets you skip the upload UI when files are already in the bucket (e.g. you re-ran a stuck upload).

Edit dishes after confirm

Once a job is `completed`, the dishes live on the restaurant detail page. To change them:

1. Go to **Restaurants** → .
2. Find the dish under its menu category.
3. Click the dish row to expand the editor.
4. Edit: name, description, price, dining_format, status, available toggle, menu/dish category.
5. Click **Save**.

Editing modifier groups on the restaurant detail page

Currently read-only there. To change modifier groups on a dish that's already saved:

1. Either: replay the menu scan and re-confirm with edited groups.
2. Or: use the backend `admin_replace_dish_modifiers` RPC directly (advanced — ping platform).

A future enhancement (planned but not yet shipped) will surface an inline edit modal on the restaurant detail page.

The dining format hint

Value	When to use	Mobile UX
— Standard — (default)	Regular plated dish. No special handling.	Normal card layout.
🍽️ **Buffet**	All-you-can-eat, flat rate.	Adds "/person" to the price label by default. Mobile shows a buffet badge.
🍷 **Course menu**	Multi-course tasting menus where the customer doesn't pick individual dishes (or picks from a fixed set per course).	Mobile shows a course badge; per-course choices render as modifier groups.
🔥 **Interactive dining**	Hot pot, Korean BBQ, fondue — meals where the customer cooks at the table.	Mobile shows an interactive-meal badge.
🥢 **Small / shared plates**	Tapas, dim sum, mezze — meals designed to be ordered communally.	Mobile shows a sharing badge.
🍢 **Sampler / platter**	Fixed selection of items served on one plate (sampler appetizer, platter).	Mobile renders bundled_items prominently.

For the vast majority of dishes, leave it at "— Standard —".

Common scenarios — full examples

Scenario A: Pad Thai with a protein choice

- **Name:** Pad Thai
- **Price:** $14, label = Exact
- **Primary protein:** chicken (the default base)
- **Dining format:** — Standard —
- **Menu category:** Canonical → "Noodles" (or restaurant's "Mains")
- **Dish category:** Pad Thai (create inline if absent)
- **Modifier groups:** One group:
  - Name: `Protein`, Single, min 1, max 1, **Card** ✓
  - Options:
    - Chicken — delta 0, Default ✓, advanced: primary_protein = chicken
    - Tofu — delta 0, advanced: primary_protein = vegan
    - Shrimp — delta +3, advanced: primary_protein = shellfish, **adds_allergens** = shellfish
    - Beef — delta +2, advanced: primary_protein = beef

Scenario B: Lunch combo (set menu)

- **Name:** Business Lunch
- **Price:** $24, label = Exact
- **Primary protein:** chicken (or whichever fits the main dish)
- **Dining format:** — Standard —
- **Bundled items:**
  - Soup of the day
  - Side salad (note: house dressing)
  - Drink (note: choice of soda or iced tea)
- **No modifier groups** (the customer doesn't pick — it's a fixed bundle)

Scenario C: Tasting menu

- **Name:** Chef's Tasting Menu
- **Price:** $850, label = **Per person**
- **Primary protein:** the dominant protein across courses (chicken/beef/etc.)
- **Dining format:** 🍷 Course menu
- **Modifier groups:** One per course where the diner picks:
  - `Course 1 — Starter`, Single, 1/1: Tuna Tartare (default), Beet Salad
  - `Course 2 — Main`, Single, 1/1: Wagyu Beef, Sea Bass
  - `Course 3 — Dessert`, Single, 1/1: Crème Brûlée, Mango Sorbet
- If a course is fixed (everyone gets the same), use **bundled_items** for that course instead.

Scenario D: Build-your-own bowl

- **Name:** Poke Bowl
- **Price:** $14, label = **From**

- **Primary protein:** fish (most common pick)
- **Dining format:** — Standard —
- **Modifier groups:**
  - `Base`, Single, 1/1: Rice (default), Greens
  - `Proteins`, Multiple, 1/3, **Card** ✓: Tuna (primary_protein=fish), Salmon (fish), Tofu (vegan)
  - `Toppings`, Multiple, 0/10: Edamame, Seaweed, Avocado, Crispy onion
  - `Sauce`, Single, 1/1: Soy (default), Spicy mayo, Ponzu

Scenario E: Tiered-pricing wings

- **Name:** Buffalo Wings
- **Price:** $8, label = **From**
- **Primary protein:** chicken
- **Dining format:** — Standard —
- **Modifier groups:** One group:
  - `Quantity`, Single, 1/1, **Card** ✓
  - Options:
    - 6 wings — **override = $8**, Default ✓
    - 12 wings — **override = $15**
    - 18 wings — **override = $20**

(Note: override replaces base price. Useful for portion-sized tiering.)

Scenario F: AYCE Korean BBQ

- **Name:** AYCE BBQ
- **Price:** $39, label = **Per person**
- **Primary protein:** beef
- **Dining format:** 🔥 Interactive dining
- **Serves:** 1 (per person)
- **Bundled items:** (optional — list what's included like "Banchan", "Steamed rice", "Lettuce wraps")
- **No modifier groups** unless guests pick specific meats (then add a `Proteins` group with Multiple selection)

Troubleshooting / FAQ

The AI extracted text but the dish names are wrong / mangled

- Click **Replay** with `gpt-4o-2024-11-20` (the higher-quality model).
- If still wrong: the source image probably has poor contrast or weird fonts. Re-upload a clearer photo.

A dish has 5 confidence and obviously should be removed

Click **Remove** on the dish card. It greys out and is skipped on save. Click **Restore** to undo before saving.

The AI created a category like "MENU" or "SECTION" — what to do?

- Change its mode from **Custom** to **Canonical** → pick the closest fit (probably "Mains").
- Or set it to **No category** if it really doesn't belong anywhere.

A modifier option name has weird characters / encoding

Just edit it. The AI sometimes mangles special characters (é, ñ, ü). Type the correct version.

I confirmed but the dishes don't show in the mobile app

- Check the dish status — `draft` dishes don't show in the consumer feed. The default status after confirm is `draft`.
- Open the restaurant detail page and change the dish status to `published`.
- Or change all dishes in batch from the restaurant publish flow (Publish section).
- Mobile cache: it may take 1–5 minutes for the change to propagate.

I accidentally confirmed with the wrong language

Confirms can't be un-done easily. Options:

- Delete the affected dishes from the restaurant detail page and replay the scan.
- Or edit the menu_categories row directly to set the right source_language_code (advanced — ping platform).

The dish category combobox doesn't show what the AI suggested

This is the yellow "AI suggested but unmatched" warning. The fuzzy-match threshold (0.7) wasn't cleared. Either:

- Pick a similar existing category from the dropdown.
- Use the **+** button to create the suggested category inline.

A restaurant has 200+ dishes — should I scan them all at once?

No. Cap each scan at ~~50 dishes (~~5 menu pages). Beyond that, AI accuracy drops and the review takes forever. Split into multiple scans by menu section.

Quick reference — keyboard shortcuts (browser)

The review UI has no custom shortcuts. Browser-native:

- **Tab / Shift+Tab** — move between fields
- **Space** — toggle checkboxes
- **Enter** — submit forms (but the Save button is a click — Enter won't fire it)

Where the code lives

For platform engineers reading this: